

Finance Dashboard UI

Frontend Developer Intern

Internship

Assignment Details

Read the following instructions carefully before starting your assignment.

Finance Dashboard UI

Objective

To evaluate your understanding of frontend development by building a clean and interactive **finance dashboard interface**.

This assignment focuses on how you design user interfaces, structure components, and handle data on the frontend. It is not expected to be production ready or backend dependent.

Note: *If you have already built a similar dashboard project, you may submit that instead. Please include a clear explanation of how it meets the requirements along with repository or deployment links.*

Key Instructions

- You do not have to follow a strict structure. Design the layout and components in a way that feels intuitive to you.
 - Keep things simple and clear. The goal is to see how you think and build, not how complex you can make it.
 - Creativity is welcomed. Small thoughtful touches in design and interaction will be appreciated.
-

Flexibility

You are free to:

- Use any frontend framework such as React, Vue, Angular or even plain JavaScript
 - Use any styling method such as Tailwind, CSS, MUI, or your own approach
 - Work with static or mock data
 - Structure the project in your own way
-

Scenario

Imagine you are building a simple dashboard for users to track and understand their financial activity.

The interface should allow users to:

- View overall financial summary
 - Explore transactions
 - Understand basic spending patterns
-

Core Requirements

1. Dashboard Overview

Create a main dashboard that includes:

- Summary cards such as Total Balance, Income, and Expenses
- At least one **time based visualization** (e.g., balance trend)
- At least one **categorical visualization** (e.g., spending breakdown)

2. Transactions Section

Display a list of transactions with details like:

- Date
- Amount
- Category
- Type such as income or expense

Include basic features such as:

- Simple filtering
- Sorting or search

3. Basic Role Based UI

You do not need to implement full RBAC or backend logic. Just simulate roles on the frontend and change UI behavior accordingly.

For example:

- Viewer can only see data
- Admin can add or edit transactions

You can switch roles using a dropdown or toggle for demonstration.

4. Insights Section

Show a few simple insights such as:

- Highest spending category
- Monthly comparison
- Any useful observation from the data

5. State Management

Handle application state properly such as:

- Transactions data
- Filters
- Selected role

You may use any approach like Context, Redux, Zustand or even simple state if well managed.

6. UI and UX Expectations

- Clean and readable design
- Works properly on different screen sizes
- Handles empty or no data cases gracefully

Optional Enhancements

These are not required but can strengthen your submission:

- Dark mode
- Data persistence (local storage)
- Mock API integration
- Animations or transitions
- Export functionality (CSV/JSON)
- Advanced filtering or grouping

Evaluation Criteria

1. Design and Creativity

Visual quality, layout decisions, and how intuitively information is presented.

2. Responsiveness

Adaptability across screen sizes and devices.

3. Functionality

Implementation of dashboard features, RBAC behavior, and interactions.

4. User Experience

Ease of use, navigation clarity, and interaction design.

5. Technical Quality

Code structure, modularity, scalability, and best practices.

6. State Management Approach

How effectively application state and UI logic are handled.

7. Documentation

Clear README including setup instructions, overview of approach, and explanation of features.

8. Attention to Detail

Handling of edge cases, UI polish, and overall completeness.

Important Note

This assignment is meant for evaluation only. You are encouraged to make reasonable assumptions and focus on showing your approach. This is not about building something perfect. It is about showing how you approach a problem and turn it into a working interface.